Relatório do Trabalho Prático I

Integração de Sistemas de Informação · Licenciatura em Engenharia de Sistemas Informáticos · Escola Superior de Tecnologia do IPCA · 2026/27

Autor: [Nome completo] · Nº [número]

Repositório Git: https://github.com/13andre/TP_ISI_2026

Integração e análise das seleções do Mundial de 2026

Conteúdo

Relatório do Trabalho Prático I 1
Integração e análise das seleções do Mundial de 2026 1
Resumo 4
1. Enquadramento 4
2. Problema 5
3. Estratégia utilizada 5
3.1 Arquitetura em três fases 5
3.2 Tecnologias 6
3.3 Modelo de dados 6
3.4 Operadores utilizados 6
3.5 Decisões de desenho 7
4. Transformações 8
4.1 fase1_csv 8
4.2 fase1_json 10
4.3 fase2_classificacao 11
4.4 fase2_alertas 14
4.5 fase3_exportacao 15
4.6 Validação cruzada entre as duas fontes 15
5. Jobs 16
6. Vídeo com demonstração 18
7. Conclusão 19
8. Referências 20

Figure 1 - Diagrama fase1_csv 8
Figure 2 - Step Metrics fase1_csv 9
Figure 3 - Diagrama fase1_json 10
Figure 4 - Diagrama fase2_classificacao.ktr 11
Figure 5 - Step Metrics fase2_classificacao.ktr 12
Figure 6 - Query 04_consulta_classificacao 12
Figure 7 - Resultado Query 04_consulta_classificacao 13
Figure 8 - Resultado Query 04_consulta_classificacao 13
Figure 9 - Diagrama fase2_alertas 14
Figure 10 Diagrama fase3_exportacao 15
Figure 11 - Step Metrics fase3_expotacao 15
Figure 12 - Query 03_consultas_validacao 16
Figure 13 - Resultado Query 03_consultas_validacao 16
Figure 14 - Diagrama main_job 16
Figure 15 - Job metrics main_job 17
Figure 16 - QR code: vídeo de demonstração 18

Resumo

Este trabalho implementa um processo ETL (Extract, Transform, Load) com o Pentaho Data Integration (Kettle) sobre os dados do Campeonato do Mundo de 2026. Dados em dois formatos (CSV e JSON) são carregados para uma base de dados SQL Server, cruzados e validados entre si. A partir deles calcula-se a classificação da fase de grupos, identificam-se alertas de resultados (goleadas) e exportam-se os resultados em CSV, JSON e XML. Um Job orquestra todo o processo, com tratamento de erro e registo em log.

1. Enquadramento

Item	Descrição
Unidade curricular	Integração de Sistemas de Informação (ISI)
Curso	Licenciatura em Engenharia de Sistemas Informáticos, Escola Superior de Tecnologia do IPCA
Ano letivo	2026/27
Equipa	[Nome(s) e número(s); indicar se o trabalho é individual ou em grupo]
Plataforma ETL	Pentaho Data Integration (Kettle) Community 10.2
Base de dados	SQL Server Express (base mundial2026)
Repositório	https://github.com/13andre/TP_ISI_2026

2. Problema

Os dados de uma competição desportiva chegam em origens e formatos diferentes: listas de seleções, calendários, resultados, eventos de jogo. Para os analisar é preciso integrá-los, validá-los e transformá-los numa forma que responda a perguntas concretas. O objetivo deste trabalho é construir um processo ETL que, para o Mundial de 2026, responda a três questões:

- Em que grupo joga cada seleção, a que confederação pertence e como ficou classificada no final da fase de grupos?
- As duas fontes de dados utilizadas (CSV e JSON) concordam entre si?
- Que jogos merecem ser assinalados como alertas de resultado (goleadas)?

Âmbito. Seleções e jogos: grupos, classificações, confederações e alertas de resultados. Os dados provêm de um dataset público (licença CC0) com 48 seleções, 104 jogos, 16 estádios, 7 fases do torneio, 601 eventos de jogo e um ficheiro JSON com o detalhe dos jogos.

Limites. O torneio já terminou, por isso os dados são estáticos e os alertas são calculados sobre os resultados finais, e não em tempo real. A classificação cobre apenas a fase de grupos (72 jogos), porque nas eliminatórias os empates são decididos por penáltis e a contagem de pontos deixa de se aplicar.

3. Estratégia utilizada

3.1 Arquitetura em três fases

Fase	Objetivo	Processos	Resultado
Fase 1 – Extract	Ler CSV e JSON e guardar em staging	fase1_csv, fase1_json	Seis tabelas stg_
Fase 2 – Transform	Validar, cruzar, calcular classificação e alertas	fase2_classificacao, fase2_alertas	Tabela classificacao_grupo e alertas
Fase 3 – Load	Persistir e exportar	fase3_exportacao	Ficheiros CSV, JSON e XML
Orquestração	Executar tudo por ordem, com erro e log	main_job	Log de execução

Os dados lidos na Fase 1 são guardados em tabelas de staging na base de dados, em vez de passarem diretamente à Fase 2. Isto torna cada fase independente (a Fase 2 começa sempre dos mesmos dados guardados), permite repetir as transformações sem voltar a ler os ficheiros e possibilita validações em SQL.

3.2 Tecnologias

- Pentaho Data Integration (Kettle) Community 10.2: Spoon (interface), Kitchen (execução de jobs).
- SQL Server Express e SQL Server Management Studio, com o driver JDBC da Microsoft (mssql-jdbc).
- Git e GitHub para o repositório.

3.3 Modelo de dados

Tabela	Origem	Linhas	Descrição
stg_teams	teams.csv	48	Seleções, grupo e confederação
stg_matches	matches.csv	104	Jogos e resultados
stg_venues	venues.csv	16	Estádios
stg_tornement_stages	tournament_stages.csv	7	Fases do torneio
stg_match_events	match_events.csv	601	Eventos de jogo
stg_match_details	real_match_details.json	104	Resultados do ficheiro JSON
classificacao_grupo	Calculada	48	Classificação final por grupo

3.4 Operadores utilizados

Operador	Onde	Função
CSV file input	fase1_csv	Leitura dos cinco CSV
JSON input	fase1_json	Extração de campos com JSONPath
Table input / Table output	Todas	Leitura e escrita na base de dados (Truncate e insert)
Select values	fase2_classificacao	Renomear campos (perspetiva casa e visitante)
Append streams	fase2_classificacao	Juntar os dois ramos (144 linhas)

Operador	Onde	Função
Formula	classificacao e alertas	Pontos, vitórias, empates, derrotas, diferença de golos, texto do alerta
Stream lookup	fase2_classificacao	Acrescentar nome, grupo e confederação
Sort rows / Group by	fase2_classificacao	Ordenar e agregar por seleção
Add value fields changing sequence	fase2_classificacao	Posição dentro de cada grupo
Regex evaluation	fase2_alertas	Validar o formato da hora
Filter rows	fase2_alertas	Horas válidas e goleadas
Database lookup	fase2_alertas	Nome das seleções a partir de stg_teams
Text file output / JSON output / XML output	fase2_alertas e fase3	Exportação
Write to log	fase2_alertas e Job	Registo de alertas e da conclusão
Start, Transformation, Write to log, Abort job	main_job	Orquestração e tratamento de erro

3.5 Decisões de desenho

- **Idempotência.** Todos os Table output usam Truncate table, para o processo poder correr várias vezes sem duplicar linhas.
- **Copy em vez de Distribute.** Quando um passo alimenta vários destinos, o movimento de dados é Copy, para cada destino receber todas as linhas.
- **Caminhos relativos.** Os ficheiros são referenciados com a variável ${Internal.Entry.Current.Directory}, para o projeto correr em qualquer computador que mantenha a estrutura de pastas.
- **Apenas fase de grupos.** A classificação usa os 72 jogos com stage_id = 1.
- **Validação cruzada.** Como existem duas fontes para o mesmo conjunto de jogos, os resultados e os nomes das seleções são comparados.

4. Transformações

4.1 fase1_csv

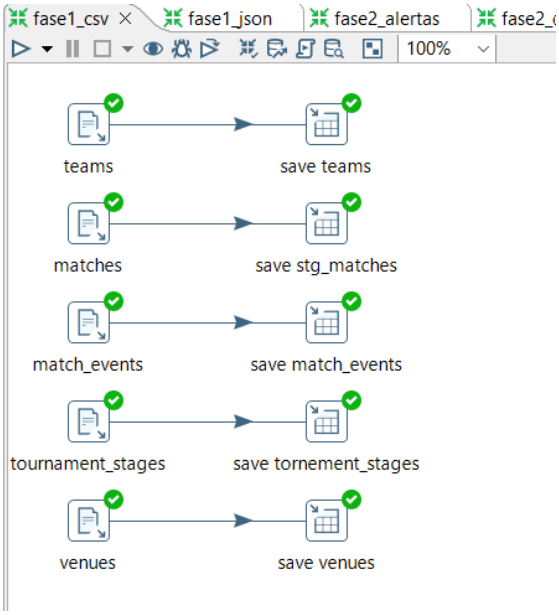


Figure 1 - Diagrama fase1_csv

A transformação tem cinco fluxos independentes e paralelos. Cada um lê um CSV com um passo CSV file input (delimitador vírgula, codificação UTF-8, cabeçalho presente) e grava-o com um Table output numa tabela de staging, com Truncate table. Os tipos dos campos foram definidos com o botão Get Fields e revistos: identificadores como Integer (comprimento 15, precisão 0) e textos como String.

Leitura	Tabela	Linhas
teams	stg_teams	48
matches	stg_matches	104
venues	stg_venues	16

Leitura	Tabela	Linhas
tournament_stages	stg_tornement_stages	7
match_events	stg_match_events	601

Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

#	Nome do step	Copia nr	Lidos	escritos	Entrada	Sa&#65533;da	Atualizados	Rejected	Erros	Ativo	Tempo	Velocidade (r/s)	Pri/ent/sai
1	tournament_stages	0	0	7	8	0	0	0	0	Finished	0.1s	91	-
2	matches	0	0	104	105	0	0	0	0	Finished	0.1s	1.129	-
3	teams	0	0	48	49	0	0	0	0	Finished	0.1s	550	-
4	match_events	0	0	601	602	0	0	0	0	Finished	0.1s	5.960	-
5	venues	0	0	16	17	0	0	0	0	Finished	0.1s	191	-
6	save teams	0	48	48	0	48	0	0	0	Finished	1.1s	43	-
7	save stg_matches	0	104	104	0	104	0	0	0	Finished	1.1s	91	-
8	save match_events	0	601	601	0	601	0	0	0	Finished	1.1s	534	-
9	save tornement_stages	0	7	7	0	7	0	0	0	Finished	1.1s	6	-
10	save venues	0	16	16	0	16	0	0	0	Finished	1.1s	14	-

Figure 2 - Step Metrics fase1_csv

4.2 fase1_json

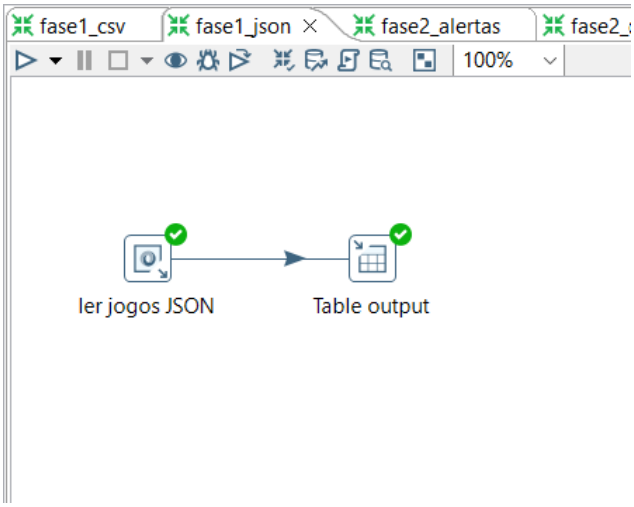


Figure 3 - Diagrama fase1_json

O ficheiro real_match_details.json é uma lista de 104 objetos, um por jogo. O passo JSON input lê o ficheiro com caminhos do tipo $[*].campo e extrai cinco campos: match_id, home_team, away_team, home_score e away_score. O resultado é gravado em stg_match_details (104 linhas). Cada jogo contém ainda listas aninhadas com os golos (home_goals e away_goals); como têm um número de elementos diferente por jogo, não podem ser lidas no mesmo passo que os campos simples, e ficam como trabalho futuro.

4.3 fase2_classificacao

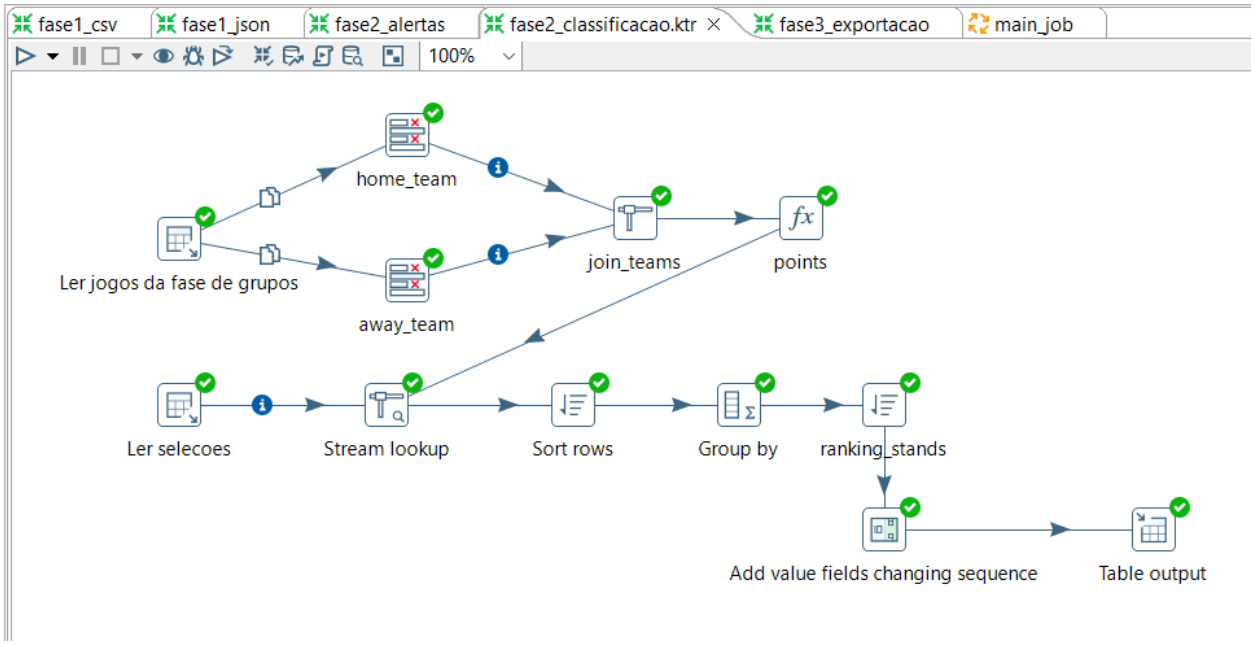


Figure 4 - Diagrama fase2_classificacao.ktr

Esta é a transformação central. A ideia é converter cada jogo em duas linhas, uma por seleção, para depois somar por seleção.

1. **Ler jogos da fase de grupos** (Table input): seleciona de stg_matches os jogos com stage_id = 1 (72 jogos).
2. **home_team e away_team** (dois Select values em paralelo, com movimento Copy): o primeiro renomeia home_team_id para team_id, home_score para goals_scored e away_score para goals_conceded; o segundo faz o inverso para a equipa visitante. Os dois ramos têm os mesmos campos pela mesma ordem.
3. **join_teams** (Append streams): empilha os dois ramos, produzindo 144 linhas (cada jogo aparece duas vezes, uma por seleção).
4. **points** (Formula): calcula, por linha, os pontos com IF([goals_scored]>[goals_conceded];3;IF([goals_scored]=[goals_conceded];1;0)), as colunas wins, draws e losses (1 ou 0) e goal_difference.
5. **Ler selecoes** e **Stream lookup**: um Table input lê team_id, team_name, group_letter e confederation de stg_teams, e o Stream lookup acrescenta esses campos a cada linha pelo team_id (192 linhas lidas, 144 escritas).

6. **Sort rows**: ordena por team_id, necessário porque o Group by só agrega linhas consecutivas.
7. **Group by**: agrupa por team_id, team_name, group_letter e confederation e calcula played (contagem), wins, draws, losses, goals_for, goals_against, goal_difference e points (somas). Saem 48 linhas.
8. **ranking_stands** (Sort rows): ordena por group_letter ascendente e depois por points, goal_difference e goals_for descendentes.
9. **Add value fields changing sequence**: cria a coluna position, que recomeça em 1 sempre que o grupo muda.
10. **Table output**: grava em classificacao_grupo (48 linhas, 12 grupos de 4, played = 3 em todas).

Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

#	Nome do step	Copia nr	Lidos	escritos	Entrada	Sa&#65533;da	Atualizados	Rejected	Erros	Ativo	Tempo	Velocidade (r/s)	Pri/ent/sai
1	Ler jogos da fase de grupos	0	0	144	72	0	0	0	0	Finished	0.2s	823	-
2	away_team	0	72	72	0	0	0	0	0	Finished	0.2s	358	-
3	Ler selecoes	0	0	48	48	0	0	0	0	Finished	0.2s	298	-
4	home_team	0	72	72	0	0	0	0	0	Finished	0.2s	358	-
5	join_teams	0	144	144	0	0	0	0	0	Finished	0.5s	267	-
6	points	0	144	144	0	0	0	0	0	Finished	0.5s	266	-
7	Stream lookup	0	192	144	0	0	0	0	0	Finished	0.6s	337	-
8	Sort rows	0	144	144	0	0	0	0	0	Finished	0.6s	242	-
9	Group by	0	144	48	0	0	0	0	0	Finished	0.6s	230	-
10	ranking_stands	0	48	48	0	0	0	0	0	Finished	0.6s	74	-
11	Add value fields changing sequence	0	48	48	0	0	0	0	0	Finished	0.7s	71	-
12	Table output	0	48	48	0	48	0	0	0	Finished	0.7s	68	-

Figure 5 - Step Metrics fase2_classificacao.ktr

04_consulta_...\André (51)) | 05_consultas_...C\André (83)) | 03_consultas_...C\André (69))

```
1  SELECT group_letter, position, team_name, played, wins, draws, losses, goals_for, goals_against, goal_difference, points
2  FROM classificacao_grupo
3  ORDER BY group_letter, position;
```

Figure 6 - Query 04_consulta_classificacao

Resultados | Mensagens

	group_letter	position	team_name	played	wins	draws	losses	goals_for	goals_against	goal_difference	points
1	A	1	South Korea	3	3	0	0	4	1	3	9
2	A	2	South Africa	3	2	1	0	4	1	3	7
3	A	3	Mexico	3	2	0	1	3	3	0	6
4	A	4	Czechia	3	1	1	1	3	5	-2	4
5	B	1	Qatar	3	2	1	0	10	2	8	7
6	B	2	Canada	3	2	1	0	9	2	7	7
7	B	3	Bosnia and Herzegovina	3	2	1	0	8	3	5	7
8	B	4	Switzerland	3	2	1	0	7	3	4	7
9	C	1	Haiti	3	2	0	1	7	3	4	6
10	C	2	Morocco	3	1	1	1	5	4	1	4
11	C	3	Brazil	3	1	1	1	4	4	0	4
12	C	4	Scotland	3	0	0	3	0	5	-5	0
13	D	1	USA	3	3	0	0	9	3	6	9
14	D	2	Australia	3	2	1	0	4	0	4	7
15	D	3	Türkiye	3	2	0	1	5	3	2	6
16	D	4	Paraguay	3	1	1	1	4	2	2	4
17	E	1	Germany	3	3	0	0	11	3	8	9
18	E	2	Ecuador	3	2	1	0	3	1	2	7
19	E	3	Côte d'Ivoire	3	2	0	1	3	3	0	6
20	E	4	Curaçao	3	1	1	1	7	3	4	4
21	F	1	Sweden	3	2	1	0	11	3	8	7
22	F	2	Netherlands	3	1	1	1	8	6	2	4
23	F	3	Tunisia	3	1	0	2	6	8	-2	3
24	F	4	Japan	3	0	2	1	3	7	-4	2
25	G	1	IR Iran	3	0	3	0	3	3	0	3
26	G	2	Egypt	3	0	2	1	3	5	-2	2
27	G	3	Belgium	3	0	2	1	2	6	-4	2
28	G	4	New Zealand	3	0	1	2	4	10	-6	1
29	H	1	Saudi Arabia	3	1	2	0	5	1	4	5
30	H	2	Spain	3	1	1	1	4	1	3	4
31	H	3	Cabo Verde	3	0	3	0	2	2	0	3
32	H	4	Uruguay	3	0	2	1	3	4	-1	2
33	I	1	Senegal	3	3	0	0	11	3	8	9
34	I	2	Iraq	3	2	0	1	9	4	5	6
35	I	3	France	3	2	0	1	7	5	2	6
36	I	4	Norway	3	1	0	2	5	10	-5	3

Figure 7 - Resultado Query 04_consulta_classificacao

37	J	1	Austria	3	2	1	0	8	4	4	7
38	J	2	Argentina	3	2	0	1	6	3	3	6
39	J	3	Algeria	3	1	1	1	7	5	2	4
40	J	4	Jordan	3	1	0	2	5	6	-1	3
41	K	1	Congo DR	3	2	1	0	5	2	3	7
42	K	2	Uzbekistan	3	2	0	1	9	4	5	6
43	K	3	Portugal	3	1	2	0	6	1	5	5
44	K	4	Colombia	3	1	1	1	2	3	-1	4
45	L	1	Ghana	3	2	1	0	3	1	2	7
46	L	2	Croatia	3	2	0	1	6	4	2	6
47	L	3	England	3	1	1	1	4	4	0	4
48	L	4	Panama	3	1	0	2	1	3	-2	3

Figure 8 - Resultado Query 04_consulta_classificacao

4.4 fase2_alertas

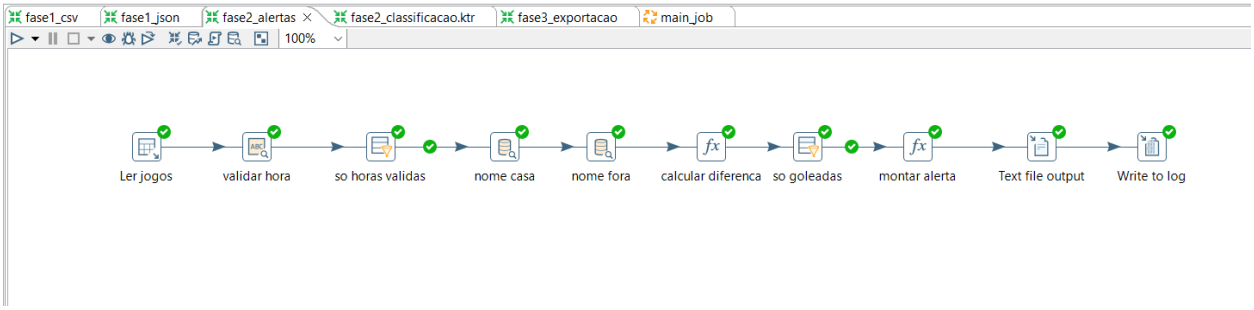


Figure 9 - Diagrama fase2_alertas

Esta transformação identifica os jogos com goleada e usa uma expressão regular para validar os dados.

1. **Ler jogos** (Table input): seleciona de stg_matches o identificador, a data (convertida para texto), a hora, os identificadores das seleções e os golos (104 linhas).
2. **validar hora** (Regex evaluation): aplica ao campo kickoff_time_utc a expressão regular abaixo, que aceita horas de 00:00 a 23:59, e guarda o resultado em hora_valida.

```
^([01]\d|2[0-3]):[0-5]\d$
```

3. **so horas validas** (Filter rows): deixa passar as linhas com hora_valida = Y. Todas as 104 linhas passaram.
4. **nome casa** e **nome fora** (Database lookup): obtêm de stg_teams o nome das seleções (home_team e away_team).
5. **calcular diferenca** (Formula): goal_diff = ABS([home_score]-[away_score]).
6. **so goleadas** (Filter rows): goal_diff >= 4.
7. **montar alerta** (Formula): compõe o texto do alerta.
8. Os alertas são gravados em export/alertas.csv (Text file output, UTF-8) e escritos no log (Write to log).

A regra de negócio, diferença de golos igual ou superior a 4, devolve quatro alertas:

Data	Alerta
2026-06-14	Germany 7-1 Curaçao
2026-06-18	Canada 6-0 Qatar
2026-06-23	Portugal 5-0 Uzbekistan
2026-06-26	Senegal 5-0 Iraq

4.5 fase3_exportacao

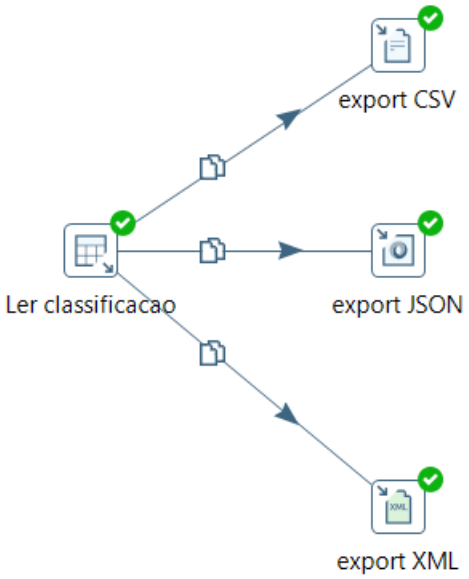


Figure 10 Diagrama fase3_exportacao

Um Table input lê a classificacao_grupo, ordenada por grupo e posição, e envia as 48 linhas (movimento Copy) para três passos de saída em paralelo: export CSV (Text file output), export JSON (JSON output, num único bloco) e export XML (XML output, com elemento de linha equipa). Os ficheiros são escritos em export/classificacao.csv, export/classificacao.json e export/classificacao.xml, com codificação UTF-8.

Execution Results

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

#	Nome do step	Copia nr	Lidos	escritos	Entrada	Sa&#65533;da	Atualizados	Rejected	Erros	Ativo	Tempo	Velocidade (r/s)	Pri/ent/sai
1	Ler classificacao	0	0	144	48	0	0	0	0	Finished	0.1s	1.059	-
2	export XML	0	48	48	0	48	0	0	0	Finished	0.2s	312	-
3	export CSV	0	48	48	0	49	0	0	0	Finished	0.2s	318	-
4	export JSON	0	48	48	0	48	0	0	0	Finished	0.2s	249	-

Figure 11 - Step Metrics fase3_expotacao

4.6 Validação cruzada entre as duas fontes

As duas fontes descrevem os mesmos 104 jogos, e foram comparadas com consultas SQL (sql/03_consultas_validacao.sql):

Verificação	Resultado
Jogos com resultado diferente entre CSV e JSON	0 linhas
Equipas da casa do JSON sem correspondência em stg_teams	0 linhas
Equipas visitantes do JSON sem correspondência em stg_teams	0 linhas
Jogos que existem só numa das fontes	0 linhas

As fontes coincidem nos 104 jogos, nos resultados e nos nomes das seleções.

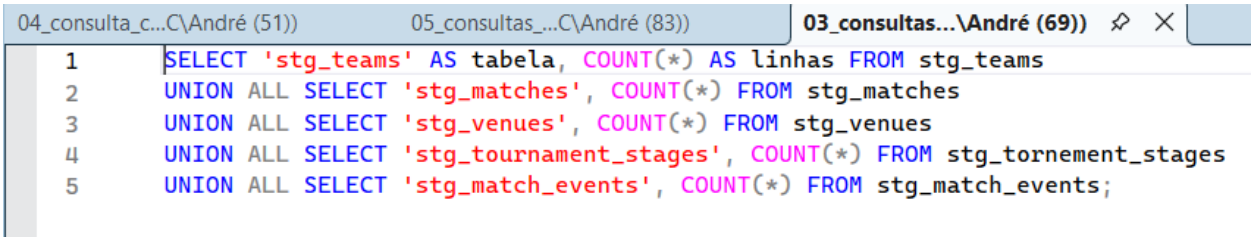


Figure 12 - Query 03_consultas_validacao



	tabela	linhas
1	stg_teams	48
2	stg_matches	104
3	stg_venues	16
4	stg_tournament_stages	7
5	stg_match_events	601

Figure 13 - Resultado Query 03_consultas_validacao

5. Jobs

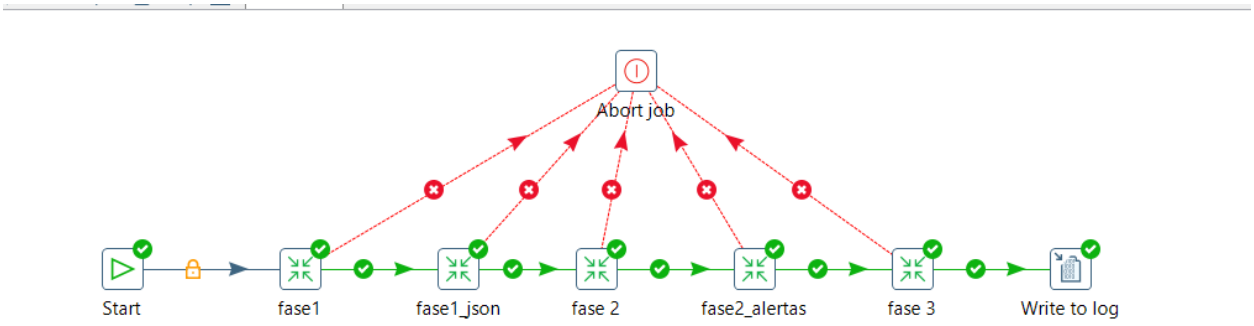


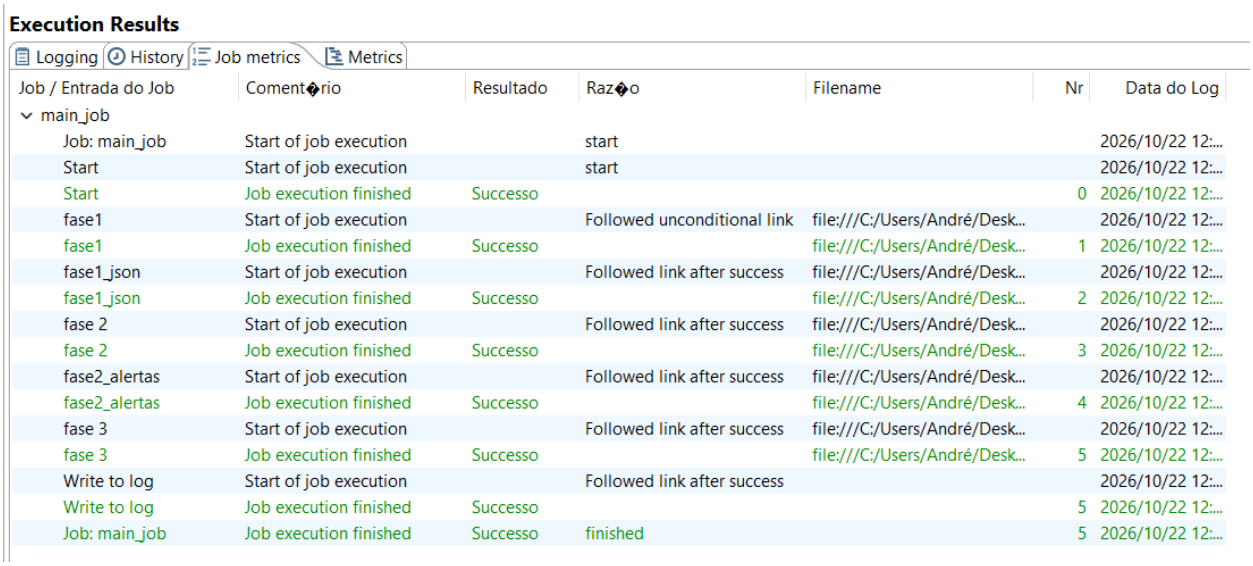
Figure 14 - Diagrama main_job

O Job main_job liga as cinco transformações por esta ordem: Start → fase1 → fase1_json → fase 2 → fase2_alertas → fase 3 → Write to log. Cada entrada de transformação referencia o ficheiro com ${Internal.Entry.Current.Directory}, para ser independente do computador.

- Os hops entre transformações são condicionais (seguir apenas se o resultado for verdadeiro).
- Cada transformação tem ainda um hop de falha para um Abort job, que termina o processo com erro se algum passo falhar.
- O Write to log final regista a mensagem «Processo ETL concluído: classificação gerada e exportada».

O Job pode ser executado no Spoon ou pela linha de comandos com o Kitchen, que guarda a execução em logs/execucao.log:

```
Kitchen.bat /file:<caminho>\workflows\main_job.kjb /level:Basic
/logfile:<caminho>\logs\execucao.log
```



Execution Results

Logging | History | Job metrics | Metrics

Job / Entrada do Job	Coment◆rio	Resultado	Raz◆o	Filename	Nr	Data do Log
main_job						
Job: main_job	Start of job execution		start			2026/10/22 12:...
Start	Start of job execution		start			2026/10/22 12:...
Start	Job execution finished	Successo			0	2026/10/22 12:...
fase1	Start of job execution		Followed unconditional link	file:///C:/Users/André/Desk...		2026/10/22 12:...
fase1	Job execution finished	Successo		file:///C:/Users/André/Desk...	1	2026/10/22 12:...
fase1_json	Start of job execution		Followed link after success	file:///C:/Users/André/Desk...		2026/10/22 12:...
fase1_json	Job execution finished	Successo		file:///C:/Users/André/Desk...	2	2026/10/22 12:...
fase 2	Start of job execution		Followed link after success	file:///C:/Users/André/Desk...		2026/10/22 12:...
fase 2	Job execution finished	Successo		file:///C:/Users/André/Desk...	3	2026/10/22 12:...
fase2_alertas	Start of job execution		Followed link after success	file:///C:/Users/André/Desk...		2026/10/22 12:...
fase2_alertas	Job execution finished	Successo		file:///C:/Users/André/Desk...	4	2026/10/22 12:...
fase 3	Start of job execution		Followed link after success	file:///C:/Users/André/Desk...		2026/10/22 12:...
fase 3	Job execution finished	Successo		file:///C:/Users/André/Desk...	5	2026/10/22 12:...
Write to log	Start of job execution		Followed link after success			2026/10/22 12:...
Write to log	Job execution finished	Successo			5	2026/10/22 12:...
Job: main_job	Job execution finished	Successo	finished		5	2026/10/22 12:...

Figure 15 - Job metrics main_job

6. Vídeo com demonstração

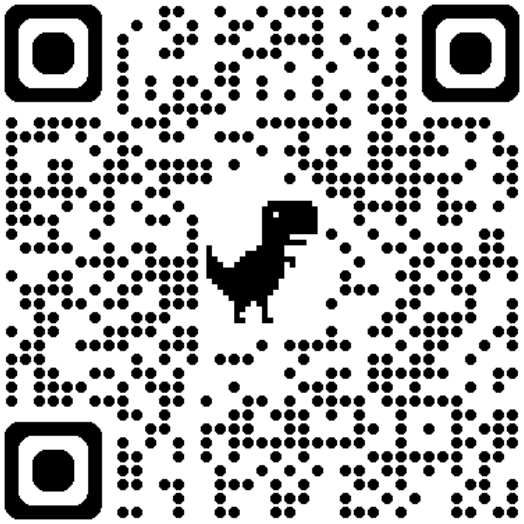


Figure 16 - QR code: vídeo de demonstração

O vídeo mostra os dados de entrada em raw/, a execução do main_job no Spoon até ao fim e os ficheiros gerados em export/.

7. Conclusão

Foi construído um processo ETL completo, que lê dados de duas fontes e formatos, valida-os por cruzamento, calcula a classificação por grupo (com confederação), identifica alertas de goleadas e exporta os resultados em três formatos, tudo orquestrado por um Job com tratamento de erro e log. A comparação entre as fontes não encontrou nenhuma discrepância, o que dá confiança nos resultados.

Critério do enunciado	Como foi cumprido
Formatos de entrada	CSV (cinco ficheiros) e JSON
Fontes de dados	Ficheiros locais e base de dados relacional (Table input e lookups)
Expressões regulares	Validação do formato da hora em fase2_alertas
Serialização JSON e XML	Leitura de JSON; exportação em JSON e XML
Joins, lookups e agrupamentos	Stream lookup, Database lookup, Append streams, Group by
Operações sobre valores	Renomear, calcular, filtrar, ordenar, concatenar
Geração de logs	Write to log e ficheiro logs/execucao.log
Orquestração	main_job com tratamento de erro
Operações sobre base de dados	Criação de tabelas, Truncate e insert, consultas de validação

8. Referências

1. Ferreira, L. e Ribeiro, Ó. (2026). Integração de Sistemas de Informação, Trabalho Prático I (enunciado). Escola Superior de Tecnologia do IPCA.
2. mominullptr. FIFA World Cup 2026 Dataset (licença CC0). GitHub. https://github.com/mominullptr/FIFA-World-Cup-2026-Dataset
3. Pentaho Data Integration (Kettle) Community 10.2, documentação oficial.
4. Microsoft. JDBC Driver for SQL Server (mssql-jdbc).
5. Microsoft. SQL Server Express e SQL Server Management Studio, documentação.